

链式前向星的例程见 10.9 节。

10.3 图的遍历和连通性

图的基本特征是点和边,图的基本算法是用搜索来处理点和边的关系。第 4 章介绍了用 BFS 和 DFS 遍历一个图,在遍历的同时也解决了图的连通性问题。BFS 和 DFS 是图论

的基本算法,本章大部分内容是基于它们的。这些算法,或者直接用 BFS 和 DFS 来解决问题,或者用其思想建立新的算法。请读者回顾 BFS 和 DFS 的内容,透彻理解并能熟练写出程序。

特别是 DFS,用递归来搜索图,比 BFS 更难理解;但是一旦理解之后,编程将十分便利。图论中的很多算法,例如拓扑排序、强连通分量等,都建立在 DFS 之上。

下面是 DFS 的示例程序,其中用 vector 邻接表来存图。用矩阵存图的 DFS 示例见第 4 章。

```
vector<int> G[N];
int vis[N];

bool dfs(int u) {
    vis[u] = 1;
    { ... ; return true; }
    { ... ; return false; }
    for(int i = 0; i < G[u].size(); i++) {
        int v = G[u][i];
        if(!vis[v])
            return dfs(v);
    }
    { ... ; }
}
```

//G[u][i]: 第 u 个结点直连的第 i 个结点
 //点的访问标志,vis = 0 表示未访问过
 //vis = 1 表示已经被正常处理过
 //vis = -1 表示正在被访问中,这在有些判断中有用
 //例如在拓扑排序中,用于判断跳出死循环
 //以 u 为起点开始 DFS 搜索
 //在本次递归中被正常访问
 //出现目标状态,正确返回
 //做相应处理,返回错误
 //u 的邻居有 G[u].size() 个
 //v 是第 i 个邻居
 //如果 v 没有访问过
 //递归访问第 v 个邻居
 //事后处理,返回正确或错误

下面用图 10.3 所示的例子来帮助读者理解 DFS 在图中的应用。这个例子故意设计成非连通图,所以从一个点出发并不能访问到所有点。

1. 求某个点的连通性

对需要的点执行 dfs(),就能找到它连通的点。例如找图 10.3 中 e 点的连通性,执行 dfs(e),访问过程见图 10.4 结点上面的数字,顺序是 ebdca。

递归返回的结果见结点下面画线的数字,顺序是 acdbe。虚线指向的结点表示不再访问,因为前面已经被访问过。

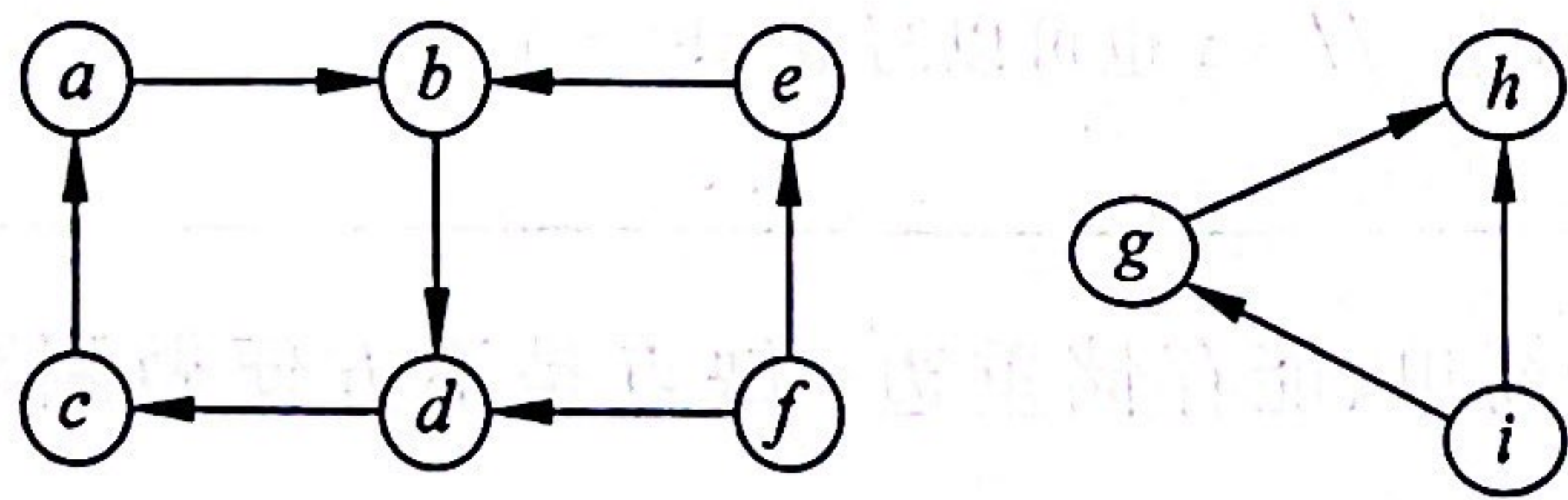


图 10.3 一个有向图例子

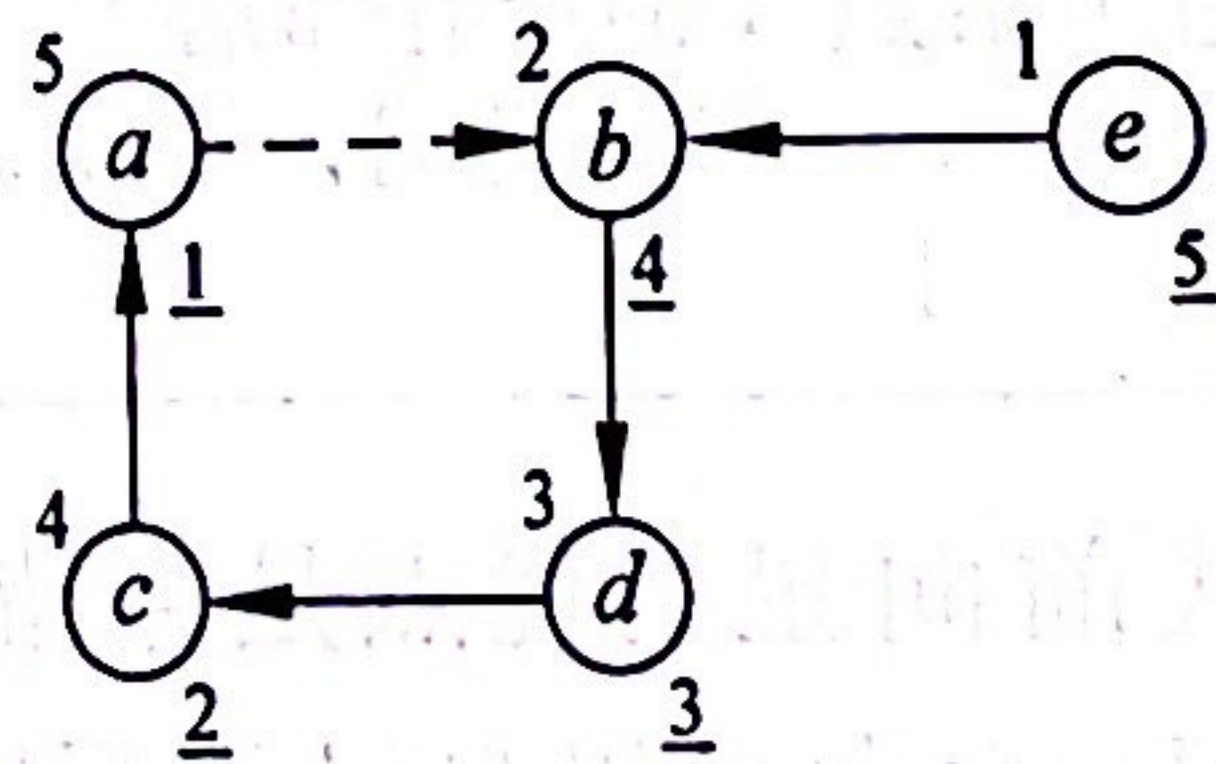


图 10.4 dfs() 的访问顺序

2. 重要概念

深搜优先生成树:上面 DFS 的结果生成了一棵树,称为深搜优先生成树(depth-first spanning tree)。

树边:树上的边称为树边(tree edge)。

回退边:虚线表示的边(a,b)称为回退边(back edge),它不在树上。

在这棵树上,从起点到其他任何一个点只有一条路径。如果是无向图生成的树,那么任意两个点之间只有一条路径。

3. 用 dfs() 处理所有点

题目经常需要处理所有的点,也可以用 dfs() 实现。其思路是想象有一个虚拟结点 v , 它连接了所有的点,那么在主程序中这样进行 dfs():

```
for(int i = 0; i < n; i++)
    dfs(i);
```

读者先自己思考,写出对图 10.3 做 dfs() 的过程,然后与下面的答案对照。

按字母顺序执行 dfs(), 访问过程见图 10.5 结点上面的数字, 顺序是 *abdcefgghi*。虚线指向的结点表示不再访问。

递归返回的结果见结点下面画线的数字, 顺序是 *cdbae fhgi*。

请读者彻底掌握本节的内容,这是本章后面内容的基础。



视频讲解

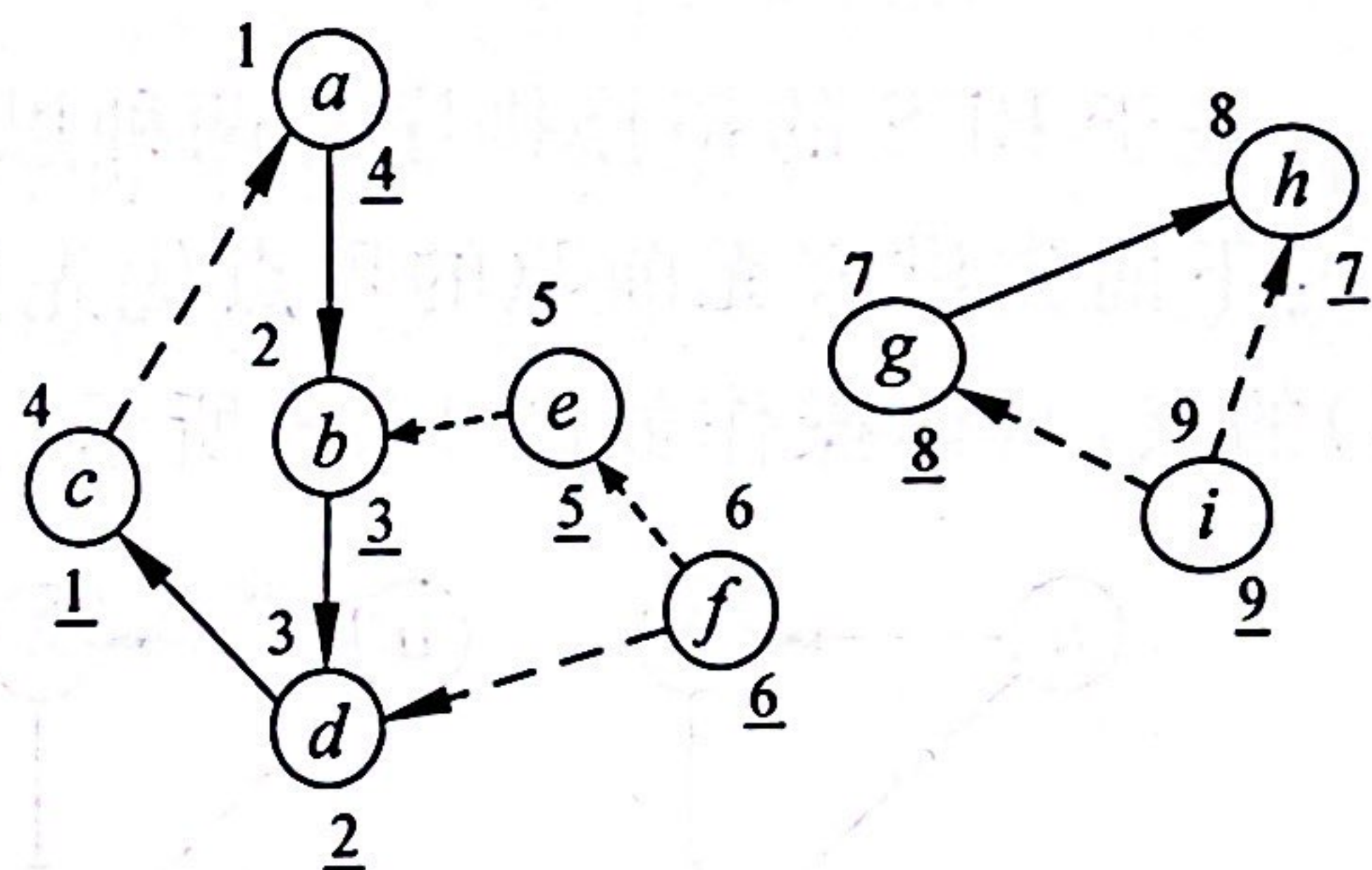


图 10.5 dfs() 访问所有点的顺序